

1. Sematic Segmentation

Sematic Segmentation nghĩa là predict class cho từng pixel trong ảnh. Tức là trong segmentation thì mỗi pixel đều có class label

Như vậy bài toán sẽ là:

Input: Image ($H \times W \times 3$)

Output: Segmentation map ($H \times W$)

Trong đó với mỗi pixel y sẽ thuộc vào 1 trong N classes



Vd trong hình có 4 class: grass, cat, tree và sky

1.1. Cách cơ bản: Sliding Window

Ý tưởng: nhìn từng pixel một $\rightarrow$ dự đoán pixel. Tuy nhiên nhiều pixel chỉ có 1 màu $\rightarrow$ khó có thể predict $\rightarrow$ nhìn các pixel xung quanh tạo thành từng patch

Lấy mỗi patch chạy classify với CNN

Một vài vấn đề:

- + Vì train độc lập từng patch nên nó sẽ không học được các patch xung quanh chỉ có học được chính patch đó

- + Quá tốn tài nguyên vì mỗi patch sẽ chạy một CNN

1.2. Nâng cấp: Convolution

Ở đây ta sẽ sử dụng Convolution network luôn. Tức là thay vì crop patch $\rightarrow$ CNN $\rightarrow$ classify thì ta dùng full image $\rightarrow$ CNN $\rightarrow$ segmentation map tức là CNN xử lý ảnh đúng 1 lần

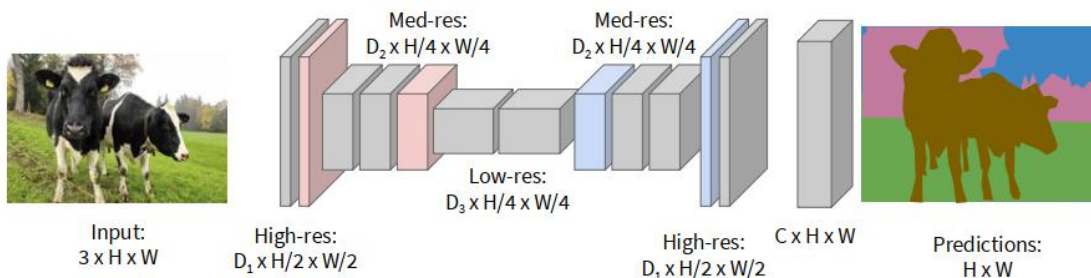
Vấn đề chính: CNN sẽ giảm size của ảnh tuy nhiên mình cần size của segmentation map có size bằng với ảnh gốc

1.3. Nâng cấp từ Convolution: Fully Convolution

Ý tưởng: Chỉ sử dụng convolutional layer không sử dụng bất kì downsampling nào

Vấn đề: Nếu chỉ dùng convolution layer thì sẽ tốn rất nhiều compute vì nó không có giảm size ngược lại còn tăng. vd $3 \times 256 \times 256$ sau 1 Conv thành $9 \times 256 \times 256 \rightarrow$ rất nhiều tham số

Nâng cấp thêm từ FC: Downsampling xong Upsampling



Downsampling: có nhiều cách đã nói từ lecture trước như: pooling, stride,

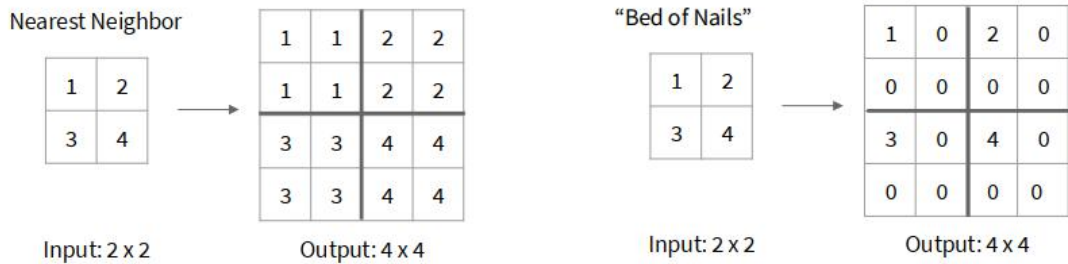
Upsampling:?

Notes: Loss của cái model này là gì? Gắn softmax loss cho từng pixel và loss tổng sẽ là tổng các loss của mỗi pixel $\rightarrow$ sẽ có ground truth của từng pixel để train

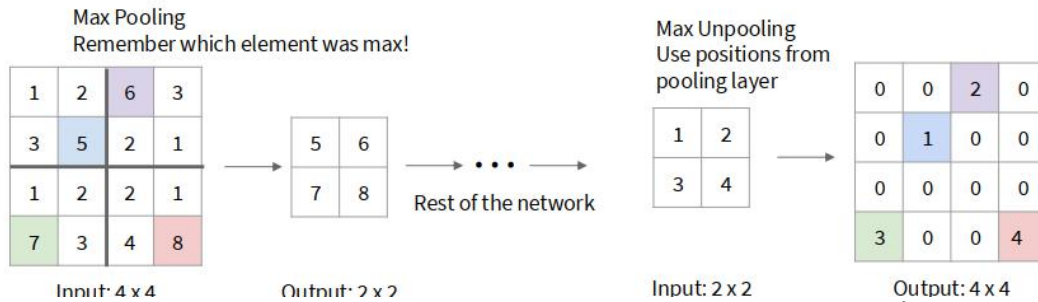
* Upsampling:

- + Nearest Neighbour: lấy số xung quanh nhiều nhất

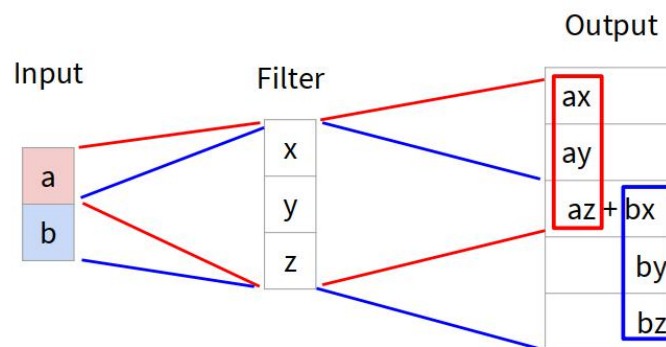
- + Bed of Nail



+ Max Unpooling: chúng ta ghi nhớ các vị trí có số lớn nhất trước khi downpooling xong đó sử dụng các vị trí đó cho unpooling

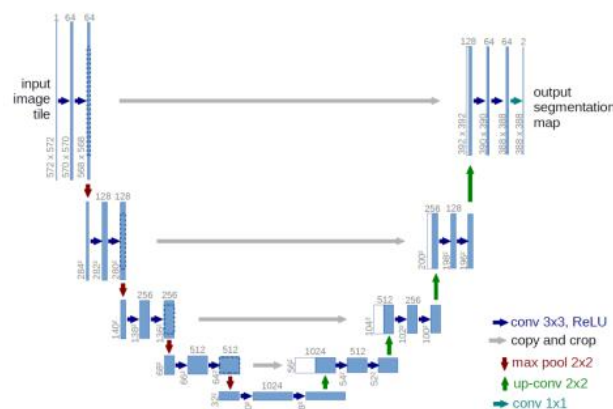


+ Learnable Sampling: này dành cho stride conv -> ta nhân lại kernel để ra lại mảng có kích thước lớn (giống như làm ngược lại với stride conv)



Vd đây là mảng 1D đối với các vùng chồng lấn thì ta cộng lại các kết quả của từng upolling

1.4. Ví dụ mô hình cụ thể: U-Net



Phía bên trái được gọi là bên Downsampling

+ Nó giúp tăng vùng nhìn thấy (vùng thu thập thông tin)

+ Giảm size của ảnh

Phía bên phải được gọi là bên Upsampling: tăng lại kích thước ban đầu

Đường màu xám: kết nối giữa downsampling và upsampling:

+ Giữ thông lại thông tin vị trí để upsampling (vd như để cho max unpooling)

* **Tóm lại:** semantic segmentation sẽ phân loại từng pixel tuy nhiên nó sẽ KHÔNG phân biệt từng object riêng. Vd có 3 con chó nó sẽ không phân biệt được 3 con chó mà nó cho cả 3 con chung 1 màu -> Bài toán mới: Instance Segmentation (trước tiên phải biết Object Detection)

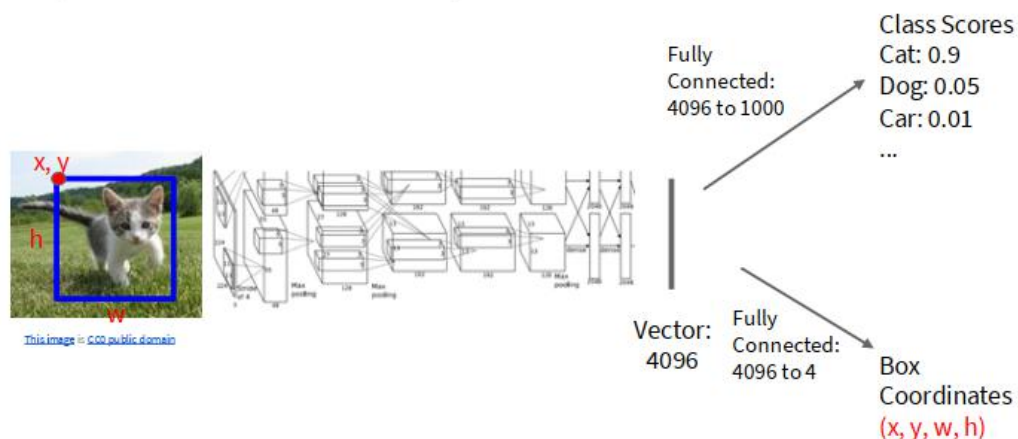
2. Object Detection

Object detection = detect object + location

Output sẽ là class của object và bounding box của object đó

2.1. Bài toán đơn giản: Single Object

Sử dụng CNN -> ra class và bbox



Loss sẽ là classification loss + bbox regression bbox dùng ground truth

Classification loss: dùng softmax loss

Bbox loss: L2 loss

2.2. Multiple Object Detection

Đơn giản chỉ cần predict được nhiều bbox

+ Sliding window: Ta lấy rất nhiều crop của ảnh rồi cho CNN classify từng crop.

Nhược điểm: phải sử dụng CNN cho rất nhiều điểm -> tốn tài nguyên và chậm

+ Region Proposals (Selective Search): tìm những điểm có khả năng

Nhược điểm: có giảm nhưng vẫn chậm. Không train chung với mô hình chính được

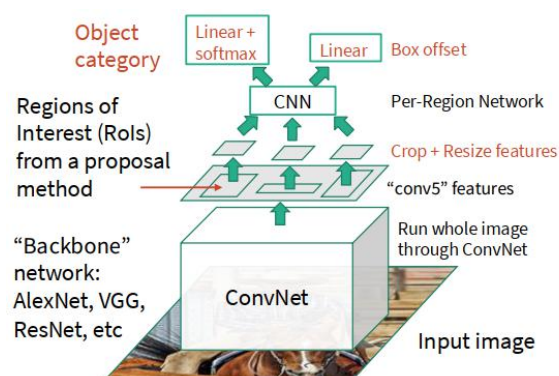
Vd: R-CNN: Image -> Selective Search -> 2000 proposals -> CNN feature extraction

-> SVM classifier -> Bounding box regression. Chậm vì phải chạy 2000 lần CNN và

phải training nhiều bước: Selective Search, CNN, SVM và Bbox regression

* Cải tiến: Fast R-CNN

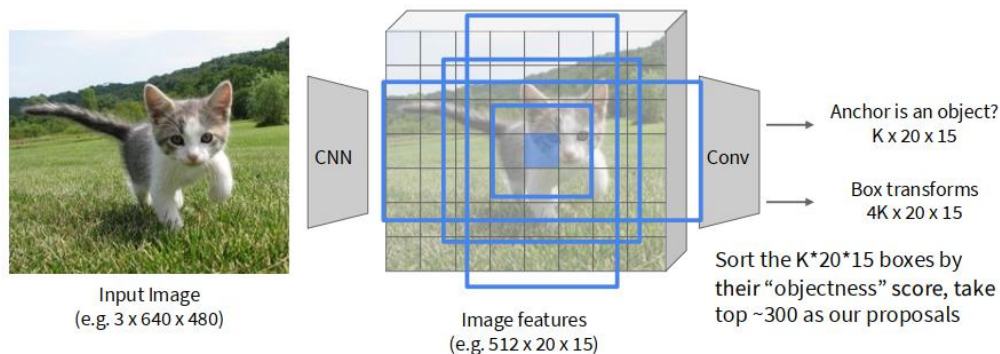
Chỉ chạy CNN 1 lần: Image -> ConvNet -> Roi -> Crop cho vào CNN -> Classifier + bbox regression



ROI ở đây giúp ta dự đoán chỗ nào có object

Cách tạo ROI dựa vào RPN (Region Proposal Network) or Selective Search

Region Proposal Network



Đây là ví dụ của RPN như ta thấy thì RPN vẫn phải dùng 1 conv để có thể dự đoán từng mảng -> vẫn phải train riêng cho cái này -> không có end2end

2.3. One-stage detector

Không dùng region proposal predict trực tiếp ra classifier + bbox

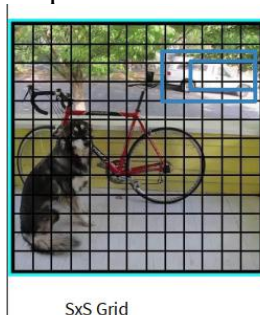
Vd: Yolo, SSD, RetinaNet

* **Yolo (You look only once)**

Ý tưởng: Chia ảnh thành từng grid và dự đoán mỗi grid

Image -> $S \times S$ grids: Một grid cell sẽ chịu trách nhiệm detect object nếu tâm object nằm trong cell đó

Bounding box prediction



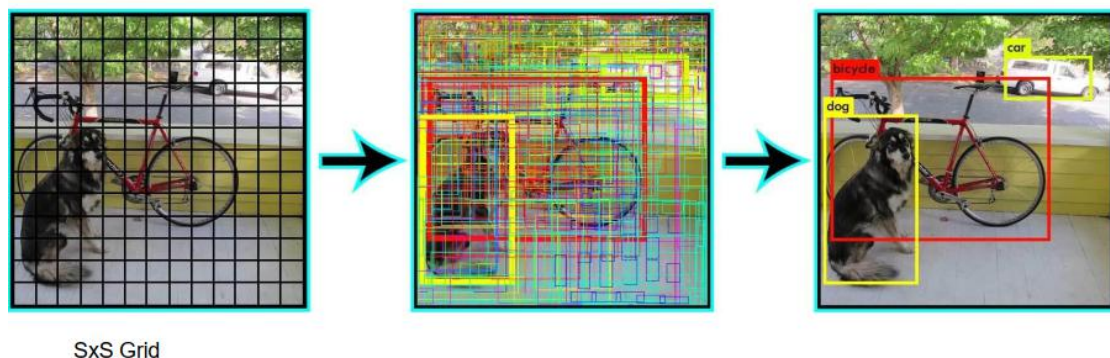
For each box output:

- $P(\text{object})$: probability that the box contains an object
- B bounding boxes (x, y, h, w)
- $P(\text{class})$: probability of belonging to a class

$B=2$

$B=2$ ở đây là số lượng object cần dự đoán là 2

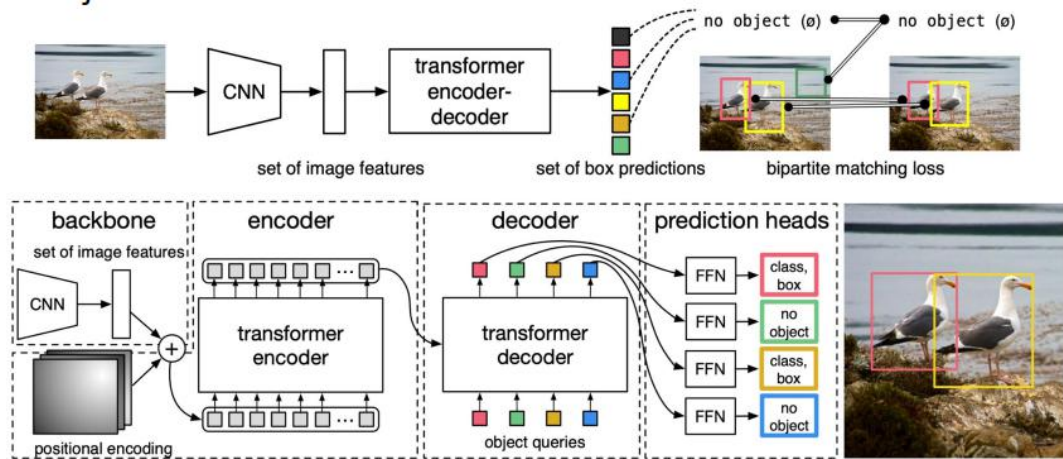
Có được bounding box xác suất của object



Yolo còn có thể dùng NMS để loại bỏ các box trùng nhau

* **DETR(Object Detection with Transformer)**

DETR không cần: region proposal, anchor boxes (không cần box mẫu), NMS(loại bỏ các box trùng nhau). Transformer sẽ trực tiếp dự đoán các bounding boxes.



Giống như transformer đã nói từ lecture trước

Note:

- + Transformer Encoder dùng self - attention để học quan hệ giữa các vùng xung quanh
- + Transformer Decoder dùng cross-attention nhận vào các query (mỗi query dùng để detect 1 object)

Tất cả các mô hình trên là fully supervise -> không dự đoán được new class

3. Instance Segmentation

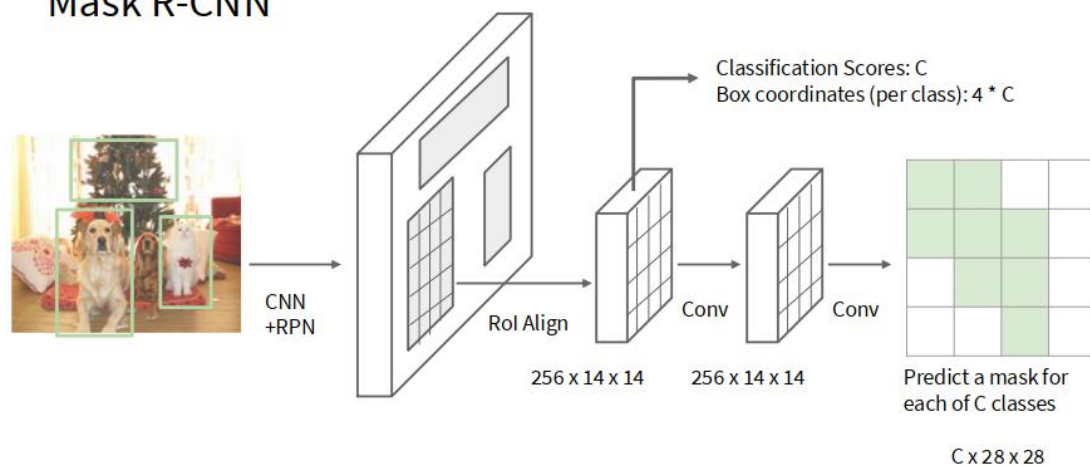
Instance Segmentation là bài toán phát hiện object + phân vùng pixel của từng object

Output: class label, bounding box, pixel mask

Mask là binary map (1 là thuộc, 0 là không thuộc)

Vd mask - RNN

Mask R-CNN



Thêm 1 layer cuối để mask các object

Loss thường là: classification loss + bbox loss + mask loss

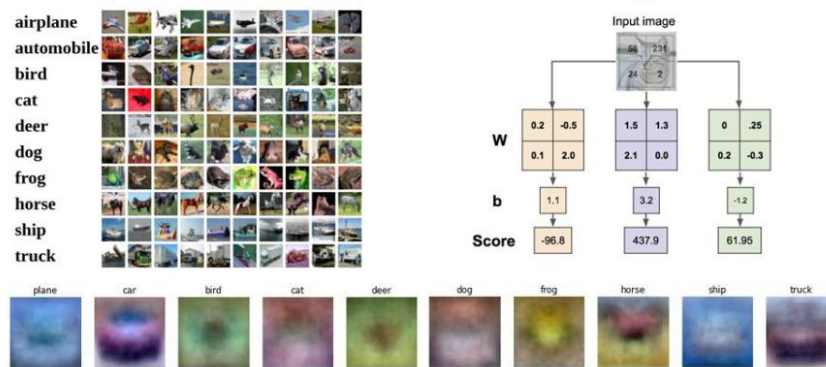
Mask loss thường là pixel-wise binary cross entropy

4. Visualization and Understanding Models

Mục tiêu: hiểu model học được gì, model nhìn vào vùng nào của ảnh, model ra quyết định dựa trên đặc trưng nào

4.1. Interpreting a Linear Classifier (Visual Viewpoint)

Interpreting a Linear Classifier: Visual Viewpoint



Nếu ta reshape W thành dạng ảnh thì ta có thể thấy template chung của object
Tuy nhiên, model rất hạn chế chỉ học được đặc điểm chung không sâu, chỉ tiết

4.2. Visualizing CNN Layers

Cách coi các layer sẽ học những gì

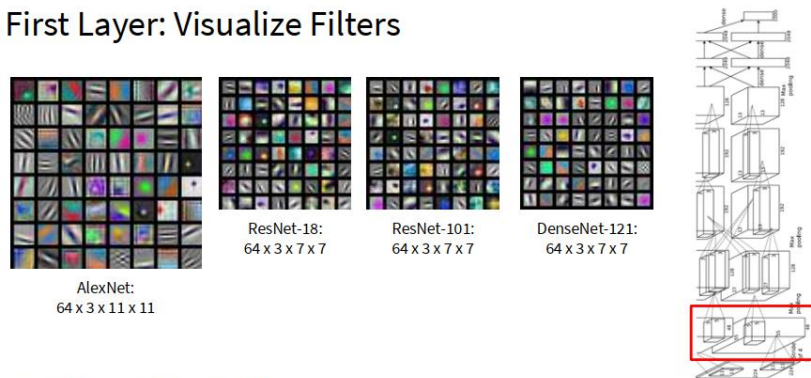
Ví dụ:

Early layer học edge, corner, color, ...

Middle layer: học object part như mắt, tai, mũi

Deep layers học semantic như mặt của con chó như nào, hình dạng chiếc xe, ...

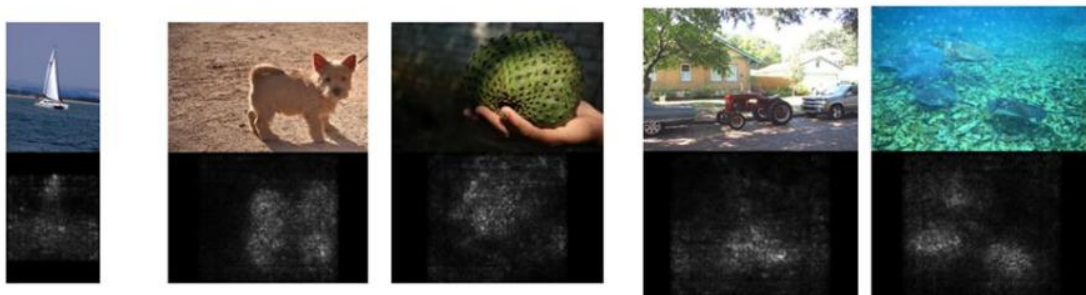
First Layer: Visualize Filters



Ví dụ như các layer đầu trong ảnh của các backbone học các cảnh các góc và phối màu của ảnh

4.3. Saliency Maps

Saliency map cho biết pixel nào quan trọng và ảnh hưởng nhiều đến prediction

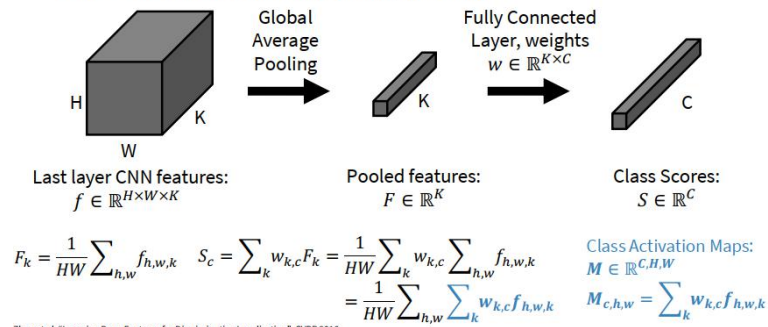


Vd: các vật thể quan trọng của từng hình pixel sẽ sáng hơn

Cách tính: ta so gradient score class của pixel chia cho gradient của ảnh $\rightarrow$ gradient càng lớn thì càng quan trọng

4.4. Class Activation Mapping (CAM)

Class Activation Mapping (CAM)



CAM cho biết vùng ảnh giúp model dự đoán class

CAM xài heatmap để đánh dấu những điểm vùng ảnh dự đoán

4.5. Grad - CAM

CAM phải dùng GAP (Global Average Pooling: mỗi feature map $\rightarrow$ 1 số trung bình)

Grad-CAM dùng gradient của class score